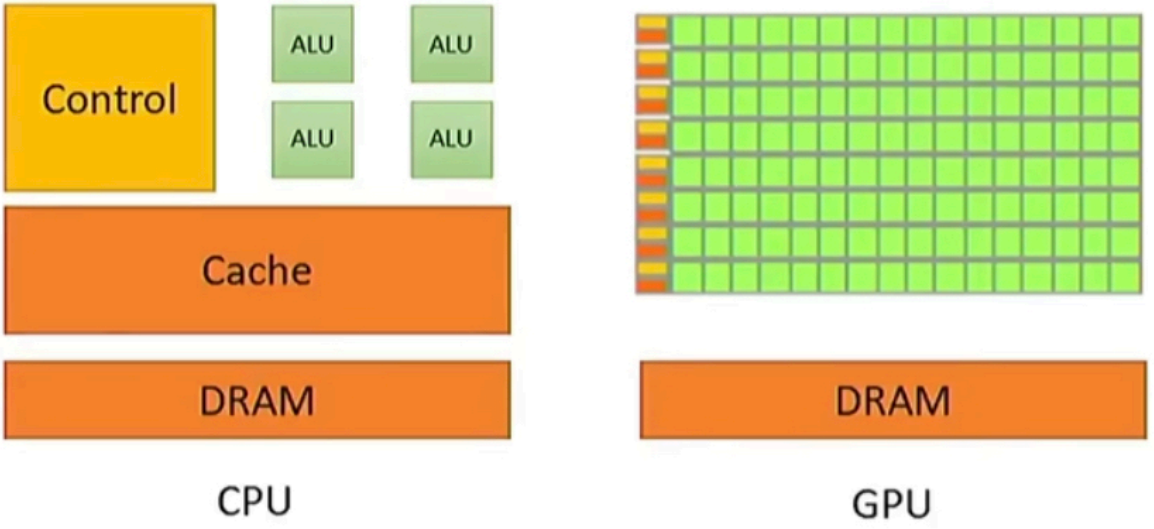
GPU 架构讲解

GPU工作原理及重要组成部分

计算单元：

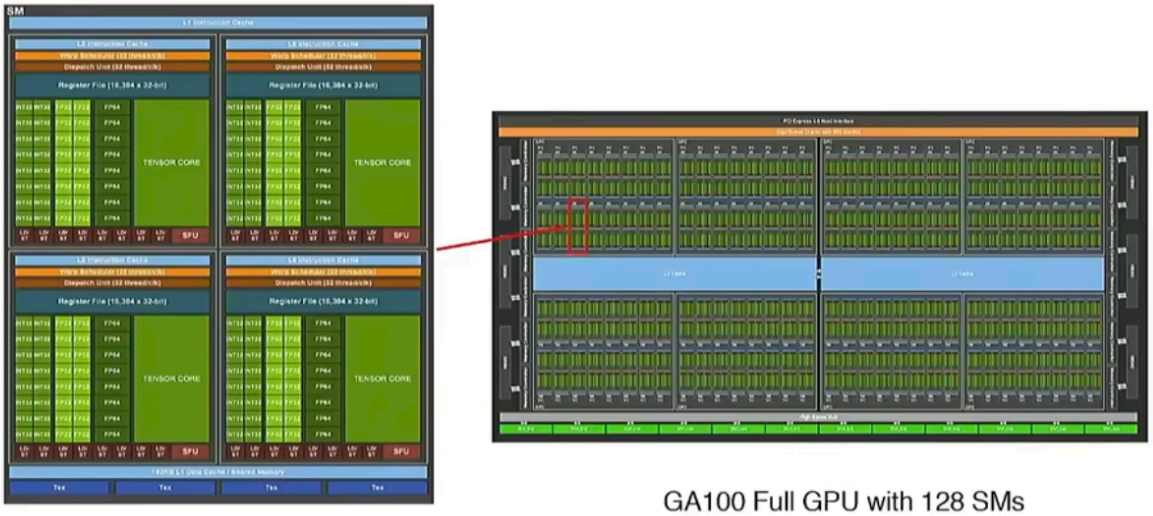


1. GPU由大量的计算单元构成（ALUs）；不像CPU，GPU中只有少量的控制单元和cache（为了支持分支）
2. 设计目标：

1. CPU：追求低延迟，快速完成当前线程任务

2. GPU：追求高吞吐，多线程快速完成所有任务，不在乎单个任务的执行时间

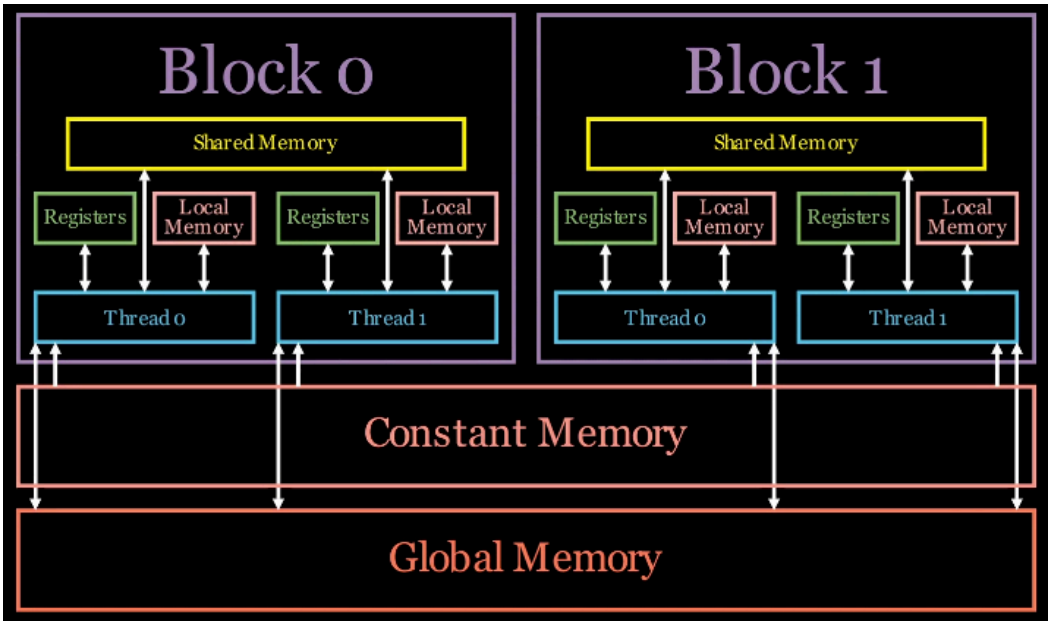
3. GPU计算单元：



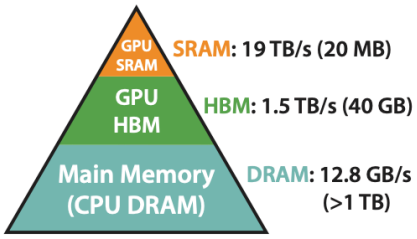
SM

1. GPU中执行大量的SM（Stream Multi-processors），每个SM中存在SP（Stream Processor）
2. 一个SM为一个原子单元，类似于CPU中的core
3. 一个SP会执行大量的线程
4. SM有很多控制逻辑，可以决定执行什么，比如分支操作
5. SPs的操作为接受相同的指令，作用于许多不同的数据
6. 所以可以执行大量的并行操作，SM作为最小可控制单元，SP可以单独进行大量计算

GPU内存：



	On/Off Chip	Access	Scope	Lifetime
Registers	On	R/W	Thread	Thread
Shared	On	R/W	Block	Block
Local	Off	R/W	Thread	Thread
Global	Off	R/W	Global	Host Controlled
Constant	Off	R	Global	Host Controlled



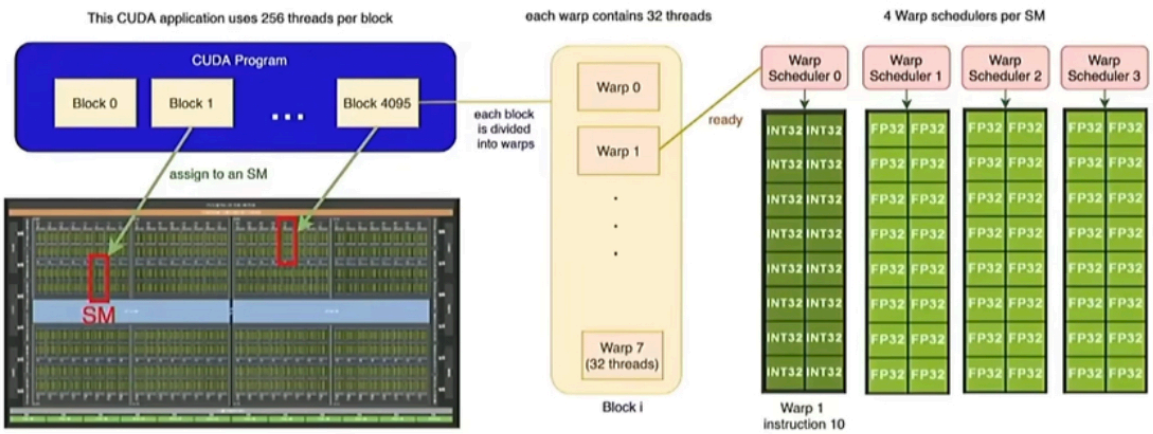
Memory Hierarchy with Bandwidth & Memory Size

THE MEMORY ACCESSES LATENCIES

Memory type	*CPI (cycles)*
Global memory	290
L2 cache	200
L1 cache	33
Shared Memory (ld/st)	(23/19)

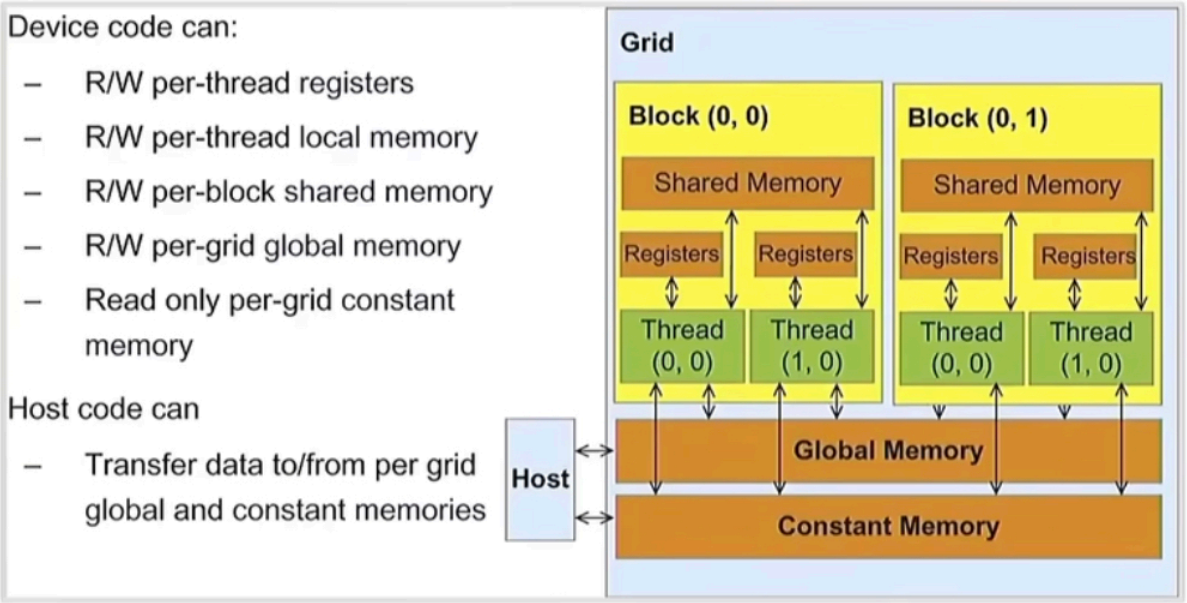
1. 每一块内存的物理位置离SM越近，它的速度越快
2. L1 cache和Shared Memory在SM中所以非常的快
3. L2 cache紧挨着SMs
4. Global Memory在芯片外，在GPU板子上；当你访问Global Memory时，你的SM如果已经完成了计算任务，则处在空闲状态
5. HBM（High Bandwidth Memory，高带宽内存）被用作Global Memory
    a. HBM特性：
        i. 海量数据存储
        ii. 提供绝对带宽：HBM 的核心指标是 带宽（Bandwidth, TB/s）。它决定了 GPU 每秒能从外部“拉”多少数据进来
        iii. 高延迟（High Latency）
6. SRAM（Static Random Access Memory，静态随机存取存储器） 被用作L1、L2、L3、L4 cache

GPU的逻辑执行模型：Simkey（Single instruction multi-thread 单指令多线程）



1. Blocks：为一组Threads的集合，它会被分配给一个SM去处理
2. Threads：一组内所有线程并行执行相同的指令处理不同的数据
3. Warp：每次将以32个连续标号的Threads为一组执行
4. 一个warp内的线程可以通过Fast Shuffle Instructions进行通信、数据交换和协作执行矩阵乘法。
    a. 同一个 Warp 内的线程直接交换彼此寄存器中的数据，传统的线程间通信通常需要通过共享内存（SRAM）进行读写；延迟极低，是 Warp 内部最快的数据交换方式
    b. 协作执行矩阵乘法可以达到远高于普通算术运算的吞吐量
5. warp是之间通过读写共享内存通信

GPU逻辑内存模型：



1. Code一般被写入到Global Memory
2. 每个thread可以访问它自己的local memory和shared memory
3. 每个block之间的跨快信息只能通过Global memory

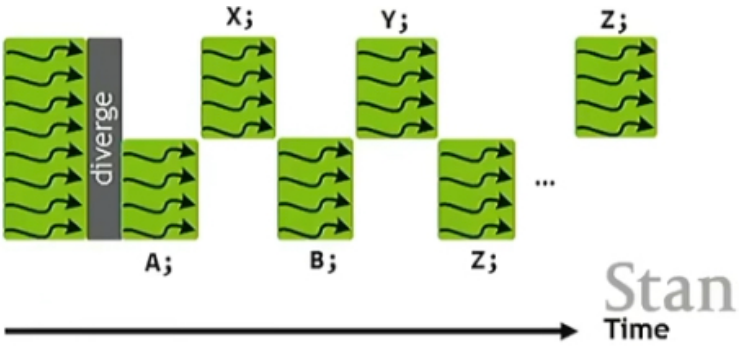
GPU 性能优化

整体目标：

确保没有必要不访问内存；对慢速的Global memory访问次数要尽可能的少

条件语句 - 不是内存问题：

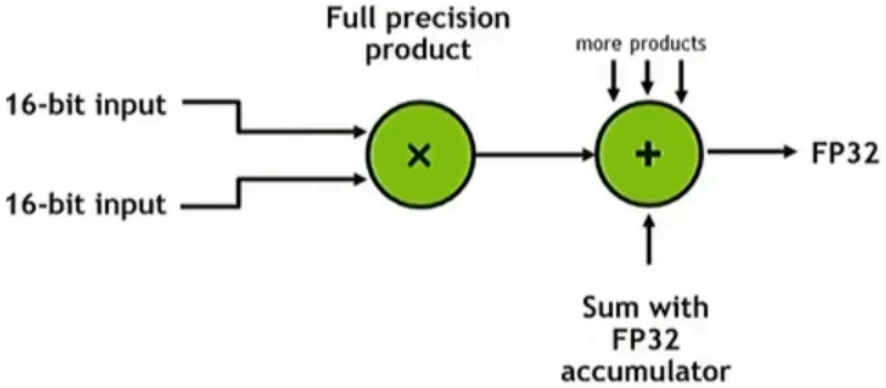
```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```



1. 问题：由于GPU的逻辑执行模型，需要每个线程执行相同的指令，所以条件语句在一个warp内部是非常具有破坏性的（导致部分线程空闲）
2. 解决方案：避免在大规模并行计算单元中放置条件语句

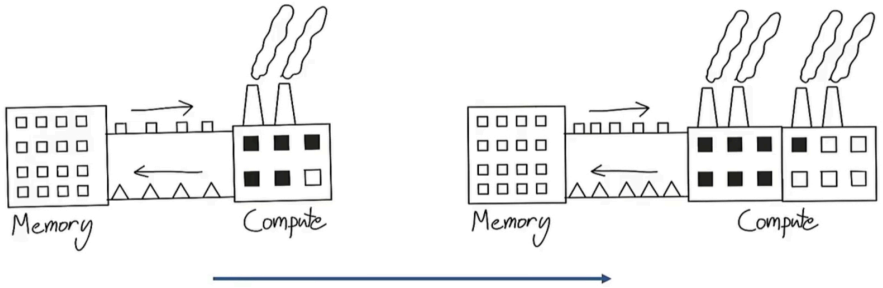
内存相关问题：

1. 降低精度 - lower precision computation（FP32, FP16, Int8）
    - 如果在所有计算中使用的位数更少，包括权重等，则所需要移动的bits就会变少
    - 这样及时从Global memory访问也不会成为主要影响性能的问题了
    - 通过降低/混合精度加速：

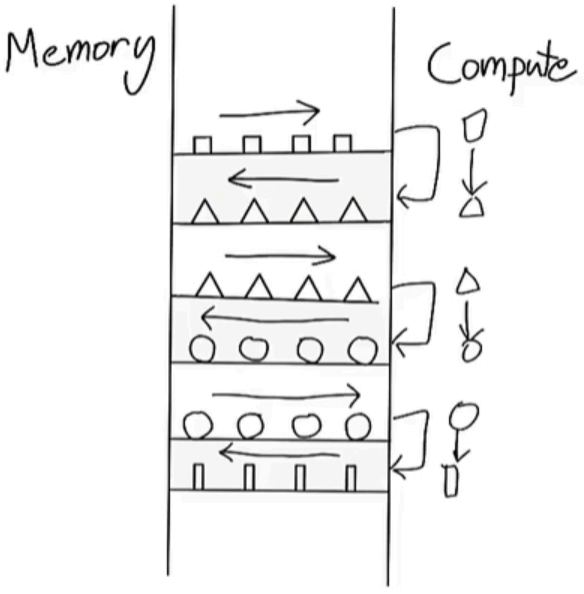


    - 可以使用16 位存储的操作：FP16/BF16
        - 矩阵乘法
        - 大多数逐点运算（Pointwise Operations）

            - 对张量（如向量、矩阵）中的每一个元素独立进行的运算
            - 例如 relu、tanh、add、sub、mul
    - 需要更高精度的操作：（FP32/BF16）
        - 将小数值加到大sum和上可能导致舍入误差
        - 归约运算（Reduction operations）
            - 例如 sum、softmax、归一化
    - 需要更宽的范围的操作（FP32/BF16）
        - 逐点运算中，若 f(x) 远大于 x ($|f(x)| >> |x|$)（例如 exp、log、pow）
        - 损失函数
2. 算子融合 - Operator fusion

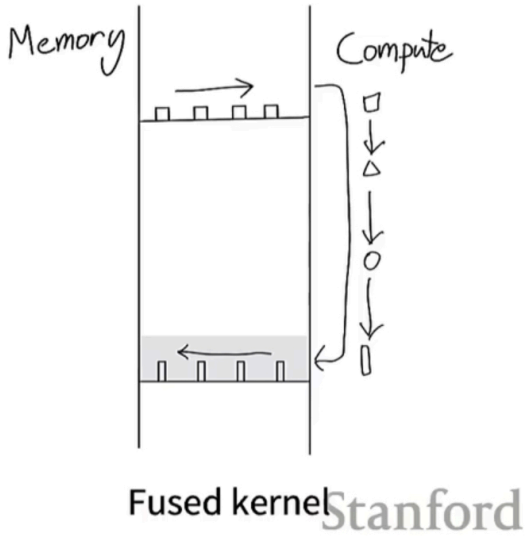


a. 问题：增加计算能力，但是由于受限于从内存到计算传输数据的速度，导致增加的计算能力单元未被用到



Naïve (non-fused)

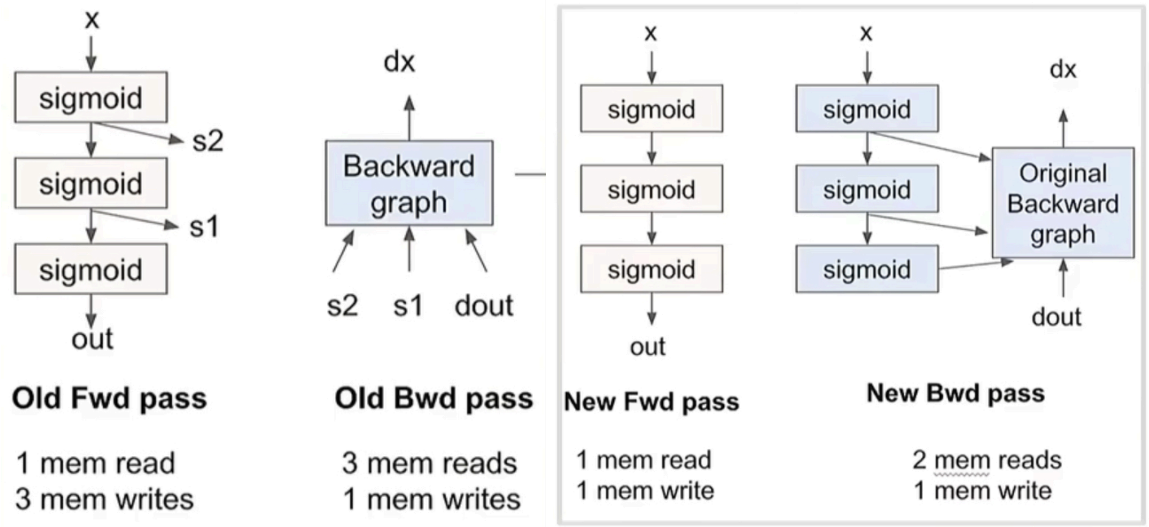
i. 造成大量内存开销



ii. 尽量在一个地方完成所有计算

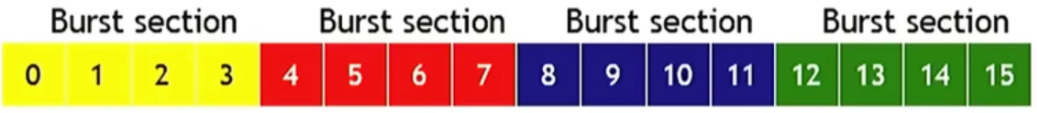
b. 尽可能多的使用torch.compile，会自动完成许多算子融合

3. 重计算：
   a. 牺牲掉一些计算量，避免进行内存访问



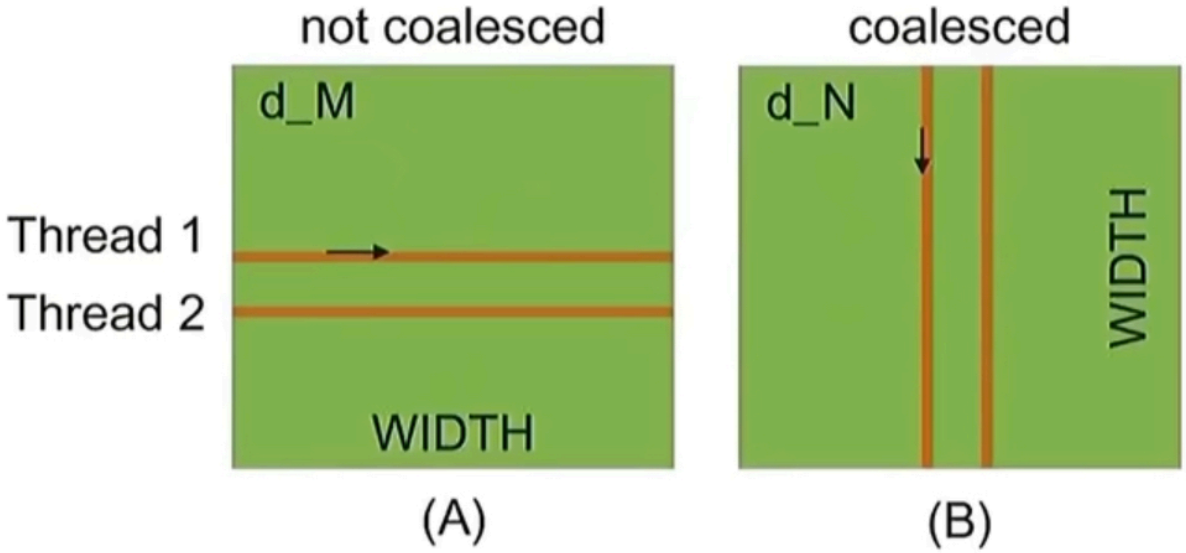
   b. 在SM和local memory总，即时计算每个sigmoid的值，并将其放入反向计算图，从而无需全局内存读取
   c. 类似gradient checkpoint和重新计算激活值以节省内存相似，但是目的不同，重计算是为了执行速度，而不仅仅是因为内存不足

4. 内存合并与DRAM：
   a. Global Memory在GPU中被称为DRAM（慢速内存）
   b. 读取某一内存位置时，不会只返回该地址信息，而是返回一个chunk

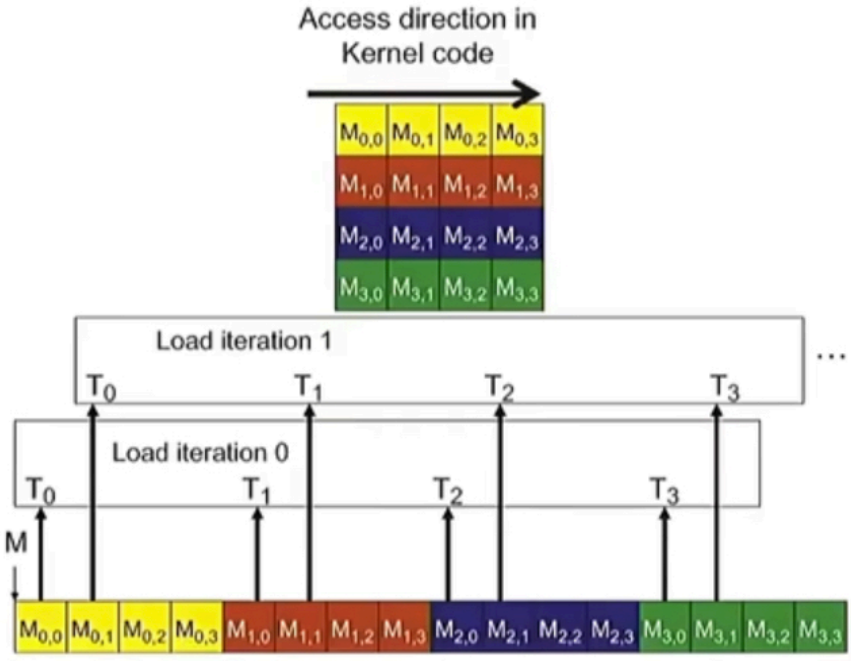


   c. 内存合并：

i. 如果一个warp中所有的线程都落入同一个burst section，则可以一次读取所有需要的值



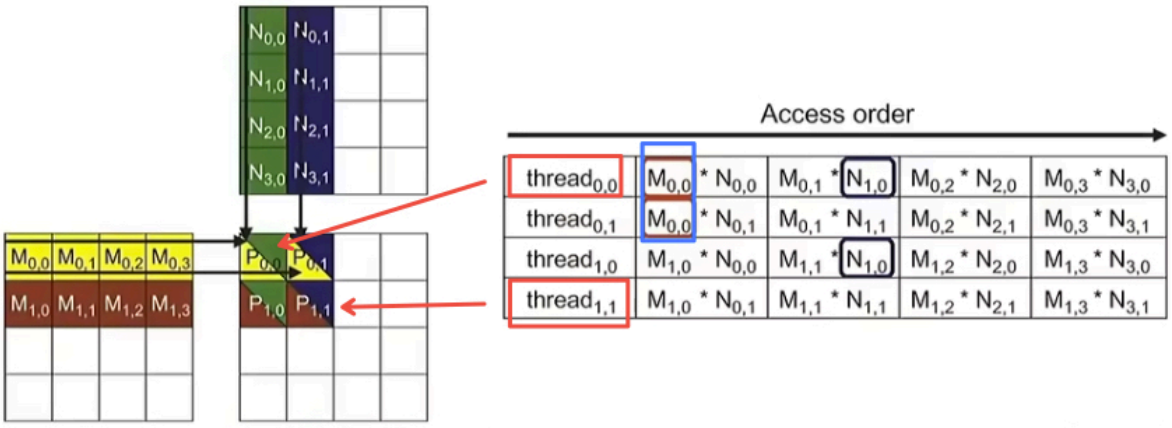
- Tips：CUDA中矩阵默认行优先（Row-Major）
  $M[i][j]$ 地址 $= base + i * WIDTH + j$
- 为什么每个线程遍历一行无法合并，但是每个线程遍历一列可以合并?
  - Badcase示例：



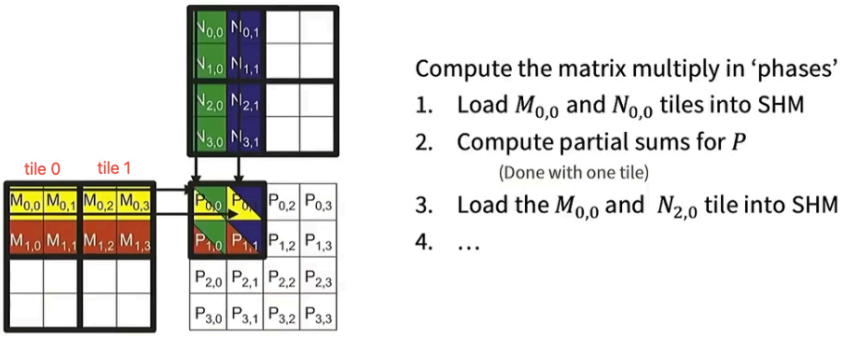
  - Timestep1：

            - Thread 1:读取M00，获取到了整个chunk（M00,M01,M02,M03），但是其余数据time step1用不到
            - Thread 2:读取M10，获取到了整个chunk（M10,M11,M12,M13），但是其余数据time step1用不到
            - …
            - 等于读取了整个内存
    - 按列遍历：
        - Timestep1：
            - Thread 1:读取M00，获取到了整个chunk（M00,M01,M02,M03），其余数据直接被其余线程使用

5. 堆叠（tiling）
    a. 核心思想：将内存访问分组以最小化必须进行的全局内存访问量
    b. 问题：例如，当我们使用质朴的矩阵乘法时，由于A矩阵的行要乘B矩阵的列，导致A无法内存合并，同时在乘法中由于存在大量的重复元素，导致了重复读取相同数据



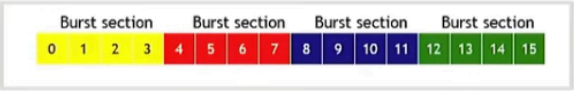
    c. 解决方案：专门花一段时间，将数据块从全局内存加载到共享内存（快速内存），在共享内存中执行大量计算 - 最大限度减少对全局内存的访问



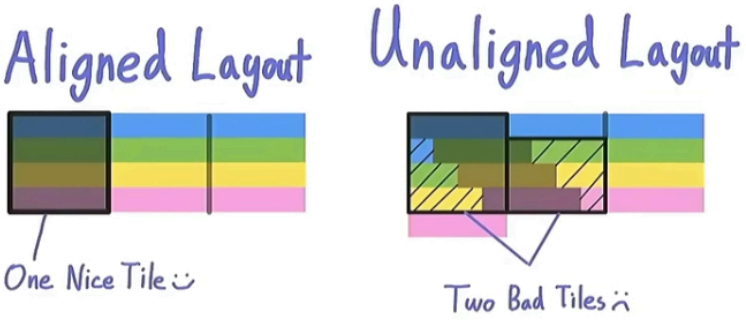
i. 拆分为子矩阵计算
ii. 同时由于单独处理数据加载，所以可以对每一个tile子矩阵都采用内存合并

d. 如果tile和矩阵大小不是burst的倍数则会导致tile和burst不对齐，从而内存访问量需要增加近一倍

Memory comes in bursts



Loading tiles are fast if bursts align with the matrix



i. 使用padding解决

总结：

1. 降低内存访问量：
   a. 内存合并
   b. 算子合并
2. 使用共享内存：
   a. 堆叠
3. 用准确性或计算换取内存：
   a. 量化
   b. 重计算